The flow between the command handler and the event handler is as follows: the command sent to the actor may trigger an event, which can modify the state. The state is passed from the command handler to the event handler—quite different from the simple actors you've seen so far. The state can be persisted by the command handler, and only then can the event handler change the state in the actor. So, when the next message arrives, the command and the current state determine the actor's behavior. You can pattern-match with both the actor's state and the command to define what the actor does. But for now, let's keep it simple and use only the command. You see how to use both state and command to define behavior in chapter 12.

In the container example, when receiving `AddCargo`, the actor tries to store a `CargoAdded` event and, if successful, changes the actor's state by adding that cargo to its list of cargo. Note that `CargoAdded` is also part of the actor's protocol and is of type `Event`.

Listing 9.4 `Container` implemented as event sourcing with Akka Persistence

```
import akka.actor.typed.scaladsl.Behaviors
import akka.actor.typed.{ ActorRef, Behavior }

import akka.persistence.typed.scaladsl.{
  Effect,
  EventSourcedBehavior
}
import akka.persistence.typed.PersistenceId

object Container {

  final case class Cargo(id: String, kind: String, size: Int)

  sealed trait Command                                              ❶
  final case class AddCargo(cargo: Cargo)
      extends Command
      with CborSerializable
  final case class GetCargos(replyTo: ActorRef[List[Cargo]])
      extends Command
      with CborSerializable

  sealed trait Event                                                ❷
  final case class CargoAdded(containerId: String, cargo: Cargo)
      extends Event
      with CborSerializable

  final case class State(cargos: List[Cargo] = Nil)                 ❸

  def apply(containerId: String): Behavior[Command] =               ❹
    EventSourcedBehavior[Command, Event, State](
      PersistenceId.ofUniqueId(containerId),
      State(),
      commandHandler = (state, command) =>
        commandHandler(containerId, state, command),
      eventHandler)

  def commandHandler(                                               ❺
      containerId: String,
      state: State,
      command: Command): Effect[Event, State] =
    command match {
      case AddCargo(cargo) =>
        Effect.persist(CargoAdded(containerId, cargo))
      case GetCargos(replyTo) =>
        Effect.none.thenRun(state => replyTo ! state.cargos)
    }
```